Random number generation

Victor Eijkhout, Susan Lindsey

Fall 2026

last formatted: November 10, 2026

1. What are random numbers?

- Not really random, just very unpredictable.
- Often based on integer sequences:

$$r_{n+1} = ar_n + b \mod N$$

- $\Rightarrow$ they repeat, but only with a long period.
- A good generator passes statistical tests.

2. Random generators and distributions

- Random device

```
1 // default seed
2 std::default_random_engine generator;
3 // random seed:
4 std::random_device r;
5 std::default_random_engine generator{ r() };
```

- Distributions:

```
1 std::uniform_real_distribution<float> distribution(0.,1.);
2 std::uniform_int_distribution<int> distribution(1,6);
```

- Sample from the distribution:

```
1 std::default_random_engine generator;
2 std::uniform_int_distribution<> distribution(0,nbuckets-1);
3 random_number = distribution(generator);
```

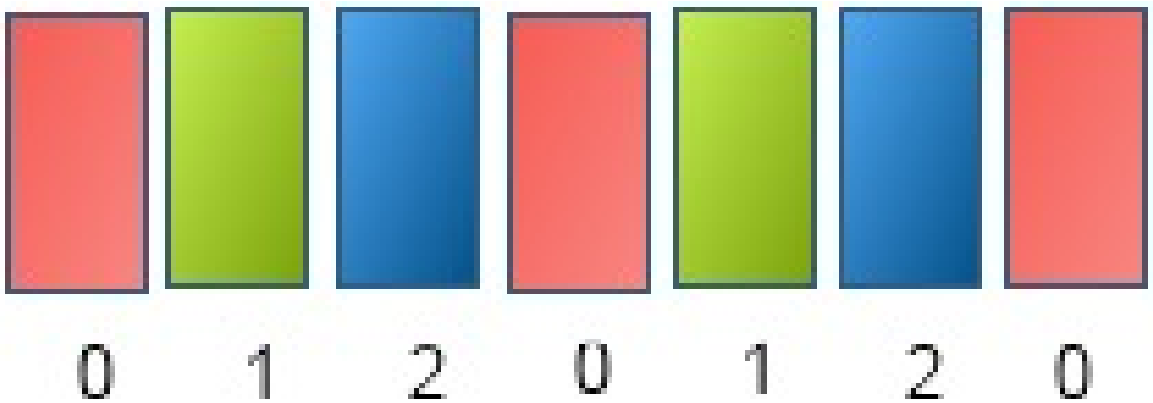
- Do not use the old C-style `random`!

3. Why so complicated?

- Large period wanted; C random has 2^{15}.
- Multiple generators, guarantee on quality.
- Simple transforms have a bias:

```
1 int under100 = rand() % 100
```

Simple example: period 7, mod 3



4. Dice throw

```
1 // set the default generator
2 std::default_random_engine generator;
3
4 // distribution: ints 1..6
5 std::uniform_int_distribution<int> distribution(1,6);
6
7 // apply distribution to generator:
8 int dice_roll = distribution(generator);
9 // generates number in the range 1..6
```

5. Poisson distribution

Poisson distributed integers:
chance of k occurrences, if m is the average number
(or $1/m$ the probability)

```
1 std::default_random_engine generator;
2 float mean = 3.5;
3 std::poisson_distribution<int> distribution(mean);
4 int number = distribution(generator);
```

6. Local engine

Wrong approach: random generator local in the function.

Code:

```
1 // rand/static.cpp
2 int nonrandom_int(int maxval) {
3   std::default_random_engine engine;
4   std::uniform_int_distribution<>
5     ints(1,maxval);
6   return ints(engine);
7 };
8     /* ... */
9   // call 'nonrandom_int' three times
```

Output:

```
1 Three ints: 15, 15,
      ↪15.
```

Generator gets recreated in every function call.

Exercise 1

What is wrong with the following code:

```
1 int somewhat_random_int(int max) {
2   random_device r;
3   default_random_engine generator{ r() };
4   std::uniform_int_distribution<> ints(1,max);
5   return ints(generator);
6 };
```

7. Global engine

Good approach: random generator static in the function.

Code:

```
1 // rand/static.cpp
2 int realrandom_int(int maxval) {
3   static  std::default_random_engine
       static_engine;
4   std::uniform_int_distribution<>
5     ints(1,maxval);
6   return ints(static_engine);
7 };
```

Output:

```
1 Three ints:
      ↪15, 98,
      ↪70.
```

A single instance is ever created.

8. Generator in a class

Note the use of `static`:

```
1 // rand/randname.cpp
2 class generate {
3 private:
4   static inline std::default_random_engine engine;
5 public:
6   static int random_int(int maxval) {
7     std::uniform_int_distribution<> ints(1,maxval);
8     return ints(generate::engine);
9   };
10 };
```

Usage:

```
1 auto nonzero_percentage = generate::random_int(100)
```